Tabular Reinforcement Learning and Planning

Thang Doi

April 8, 2026

Outline

1. Introduction
2. Value-based Approach
3. Dynamic Programming
4. Monte Carlo Method
5. Temporal Different Learning
6. Importance Sampling in off-policy Learning
7. N-step Bootstrap
8. Lambda Trace
10. Exploration and Exploitation
11. Planning and Dynamic model

Introduction to Reinforcement Learning

Overview in RL

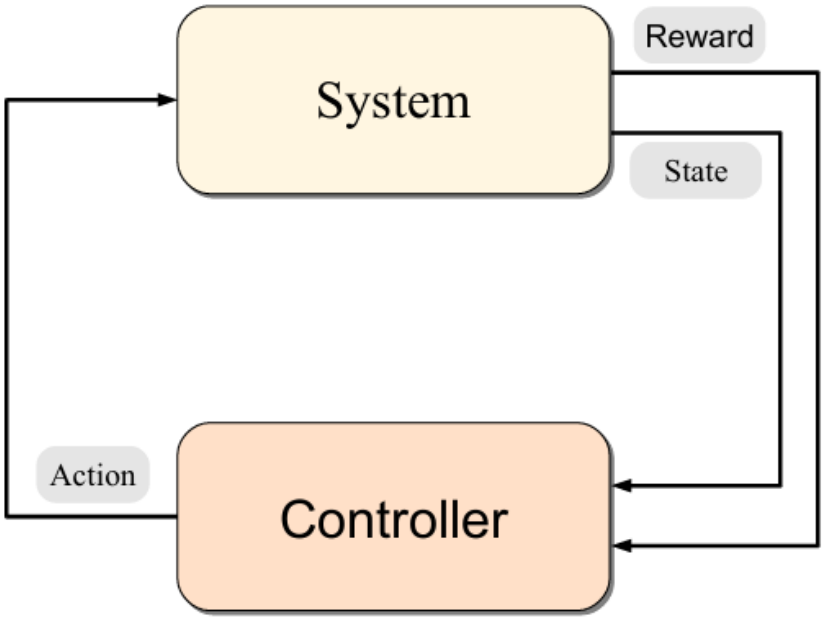


Figure: The basic reinforcement learning scenario

- **Supervised Learning and Unsupervised** learn from data observate provide from human.
- **Reinforcement Learning** is a kind of interaction learning to make sequential decisions to control a system (e.g., environment around a robot,...) to maximize a goal objective over the long term.

Component in RL

- **State**: represents the information observation of an agent. State often assigned with full observation and partial observation.
- **Action**: the response signal of an agent to the system.
- **Reward**: the signal of the environment's response to an agent. Reward decided the goal of an agent.
- **Policy**: a mapping decision of the agent making the agent to get an action from observation in the state.
  $\Longrightarrow$ Reinforcement Learning is a process of learning to make decisions over a space of state, action, and reward to reach the goal.

Markov Decision Process (MDP)

- **MDP** is a mathematical model to express the space decision of an agent over state, action, and reward.

Definition (Markov Decision Process – *Sutton & Barto 2018*)

A **Markov Decision Process** is a tuple $(\mathcal{S}, \mathcal{A}, p, \gamma)$, where

- $\mathcal{S}$ is the set of all possible states
- $\mathcal{A}$ is the set of all possible actions (e.g., motor controls)
- $p(r, s' \mid s, a)$ is the joint probability of a reward r and next state s', given a state s and action a
- $\gamma \in [0, 1]$ is a discount factor that trades off later rewards to earlier ones

Markov Property

Definition (Markov Property)

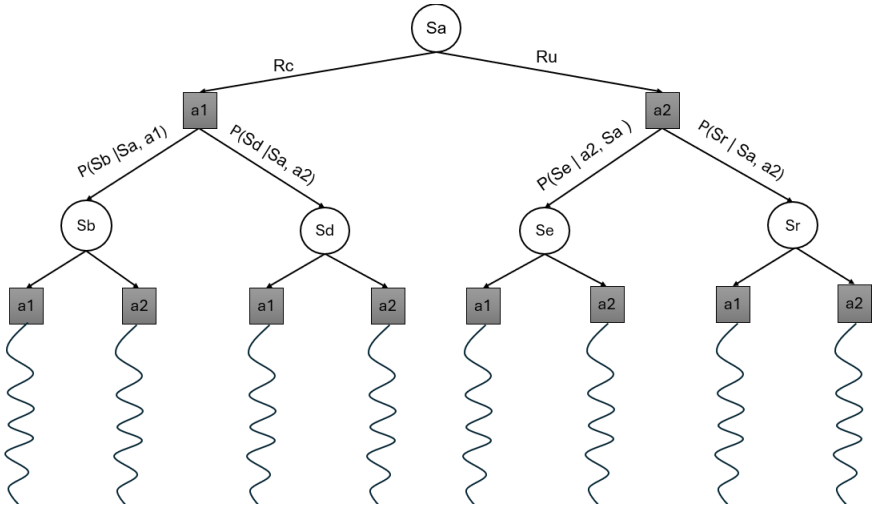
Consider a sequence of random variables, $\{S_t\}_{t \in N}$ indexed by time. A state s has the Markov property when for states $\forall s' \in S$

$$p(St + 1 = s' | S_t = s) = p(St + 1 = s' | h_{t-1}, S_t = s)$$

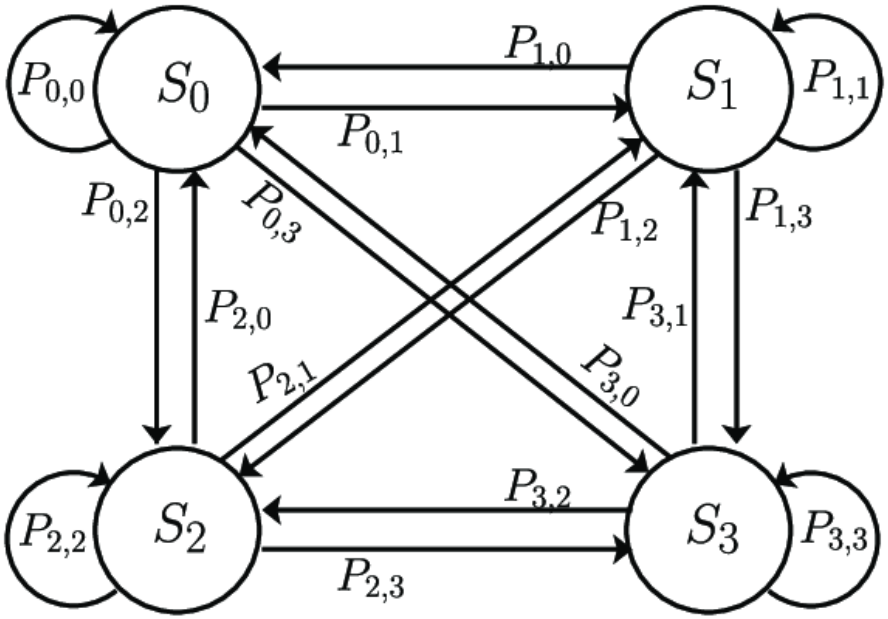
for all possible histories $h_{t-1} = S1, ..., S_{t-1}, A_1, ..., A_{t-1}, R_1, ..., R_{t-1}$

Markov property: *The future is independent of the past given the present*

Example of Markov Decision Process



(a) Infinite Markov Decision Process



(b) Finite Markov Decision Process

Figure: Examples of Markov Decision Processes

Formalizing the RL Objective

- From the MDP perspective, the objective of an RL agent is to solve the MDP — that is, to find the optimal Markov decision trajectory:

$$(s_0, a_0, \ldots, s_{t-1}, a_{t-1}, s_t) \in (S_0 \times A_0 \times \ldots \times S_{t-1} \times A_{t-1} \times S_t)$$

- The optimal Markov decision trajectory maximizes the discounted cumulative reward (or return value):

$$G_t = \sum_{k=0}^{T-t} \gamma^k R_{t+k}$$

  where $\gamma \in [0, 1]$ is the discount factor that makes immediate rewards more significant than long-term rewards.
- However, a reinforcement learning agent does not explicitly search for the optimal trajectory. Instead, it learns a policy that defines the agent's behavior in the environment.

Task in Reinforcement Learning

- **Episode**: a task has finite trajectory and terminal state
  EX: Robot in a maze, LLM agent interacts with people.
- **Continual**: a task has an infinite trajectory and does not have a terminal state. In this task, we often have a limited number of steps.
  Ex: Robot control.

Categorical Agent in RL

How agents learn to find a behaviour policy maximizing $E[G_t]$?

- **Value Aspect**: Agent can learn by estimating the expected cumulative reward $E[G_t|S = s]$ at each state, and using this value to find an optimal policy. How to estimate value?
  - Tabular method:

$$E[G_t|S = s_t] = E[R_t + \gamma R_{t+1} + ... + \gamma^T R_T|S = s_t]$$

$$E[G_t|S = s_t] = R_t + \gamma E[R_{t+1} + \gamma R_{t+2} + ...\gamma^T R_T]$$

$$E[G_t|S = s_t] = R_t + \gamma E[G_{t+1}] \ (dynamic\ formula)$$

    Dynamic Programming, Monte Carlo Method, Temporal Difference.
  - Deep Learning method:

$$E[G_t|S = s_t] \sim f(< R_t, O_t >)$$

    where O_t is the observation information from state s_t.

Categorical Agent in RL

- **Policy Aspect** Learning a stochastic policy to make a decision action.

$$\pi(a|s) = p_\pi(a|s)$$

$$\sum_{a \in A} \pi(a|s) = 1$$

  $\pi(a|s)$ often learned by using a neural network. And we called it be policy gradient.
- **Model Aspect**: A model predicts what the environment will do next: $P(s, a, s') \approx p(S_{t+1} = s'|S_t = s, A_t = a)$ (transition model), $R(s, a) \approx E[R_{t+1}|S_t = s, A_t = a]$ (reward model).
  - Model-free: no need model, directly learning by interacting with the environment.
  - Model-based: learning a model environment and using it to learn an optimal policy.

Value-based Approach

Value Function

- State value function $V_\pi(s)$ gives the expected long-term return starting from state s and following policy π:

$$\begin{aligned} V_\pi(s) &= \mathbb{E}[G_t \mid S_t = s, \pi] \\ &= \mathbb{E}[R_{t+1} + \gamma V_\pi(S_{t+1}) \mid S_t = s] \\ &= \sum_{a \in A} \pi(a \mid s) \sum_r \sum_{s' \in S} p(r, s' \mid s, a) \left[r + \gamma V_\pi(s') \right] \end{aligned}$$

- State-action value function $Q_\pi(s, a)$ represents the expected return starting from state s, taking action a following the policy π:

$$\begin{aligned} Q_\pi(s, a) &= \mathbb{E}\big[R_{t+1} + \gamma V_\pi(S_{t+1}) \mid S_t = s, A_t = a\big] \\ &= \mathbb{E}\big[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a\big] \\ &= \sum_r \sum_{s'} p(r, s' \mid s, a) \left(r + \gamma \sum_{a'} \pi(a' \mid s')\, q_\pi(s', a') \right) \end{aligned}$$

Bellman Equation

Definition (Bellman Expectation Equations)

Given an MDP, $M =< S, A, P, r, \gamma >$ for any policy $\pi : S \times A \to [0, 1]$, the value functions obey the following expections equations:

$$V_\pi(s) = \sum_{a \in A} \pi(s, a)[r(a, s) + \gamma \sum_{s' \in A} p(s'|a, s) v_\pi(s')]$$

$$Q_\pi(s, a) = r(s, a) + \gamma \sum_{s' \in S} p(s'|a, s) \sum_{a' \in A} \pi(a'|s') Q_\pi(s', a')$$

relationship between state and state-action value:

$$V_\pi(s) = \sum_{a \in A} \pi(a|s) Q_\pi(s, a), \forall s$$

Bellman Optimality Equation

Definition(Bellman Optimality Equations)

An **optimal policy** π^* is the policy that yields the highest expected return from each state, i.e.,

$$\pi^* = \arg\max_{\pi} V^{\pi}(s), \quad \forall s \in \mathcal{S}$$

and the corresponding optimal value functions are

$$V^*(s) = \max_{\pi} V^{\pi}(s), \quad Q^*(s,a) = \max_{\pi} Q^{\pi}(s,a).$$

Bellman Expectation Operator

- From the Bellman expectation equation, we define the Bellman operator

Definition(Bellman Expectation Operator)

Given an MDP, $M =< S, A, p, r, \gamma >$, and let V_S be the space of value function over state space S. For any policy $\pi : S \times A \to [0, 1]$, we define the bellman expectation operator $T^\pi : V \to V$ as:

$$(T^\pi V)(s) = \sum_a \pi(s, a)[r(s, a) + \gamma \sum_{s'} p(s'|a, s)V(s')], \forall V \in V_S$$

Properties of the bellman operator T^π

1. It has one unique fixed point V_π

$$T^\pi V_\pi = V_\pi$$

   a fixed poiny of an operator T satisfy that $Tx = x$ with $x \in X$
2. T^π is a γ-contraction

$$||T^\pi V - T^\pi||_\infty \leq \gamma ||V - U||_\infty, \forall U, V \in V_s$$

3. T^π is monotonic:

$$\forall U, V \text{such that } U \leq V, \text{then } T^\pi U \leq T^\pi V$$

- The concept of fixed point and contraction can read detail in the functional analysis concept.
- The three properties gaurantee that when evaluate the state function of a policy, it will convergence to the real-value function of this policy.
- **Policy evaluation** is an example of using Bellman expectation operator to evaluate the value function.

Bellman Optimality Operator

- Bellman optimality operator replace mean operator by a max operator

Definition(Bellman Optimality Operator)

Given an MDP , $M =< S, A, p, r, \gamma >$, and let V_S be the space of real-valued function over state S. We define the bellman optimality operator $T^* : V \to V$ as:

$$(T^*V)(s) = \max_a [r(s,a) + \gamma \sum_{s'} p(s'|a,s)V(s^{'})], \forall V \in V_s$$

- The bellman optimality operator also have the same three properties like bellman expectation operator
- The bellman optimality operator used in several algorithms such as value-iteration, or Q-learning.

Dynamic Programming

Dynamic Programming

- Value-based based on the Bellman equation to solve the reinforcement learning problem.
- The Bellman equation is used to solve two main problems in value-based.
  - **Policy evaluation** (or policy prediction) problem estimate V_π or Q_π of the current policy.
  - **Policy improvement** problem purpose: to improve the current policy based on value V_π or Q_π from the policy evaluation.
  - The process is iterative between policy evaluation and policy improvement be called the control problem.
- Dynamic programming is the simplest algorithm to solve the RL problem, given a perfect model of the environment. And consist two important parts **policy evaluation** and **policy improvement**.
  - **policy iteration** separate the two process policy evaluation and policy improvement.
  - **value iteration** integrates two process policy evaluation and policy improvement in one iteration.

Policy Iteration (1)

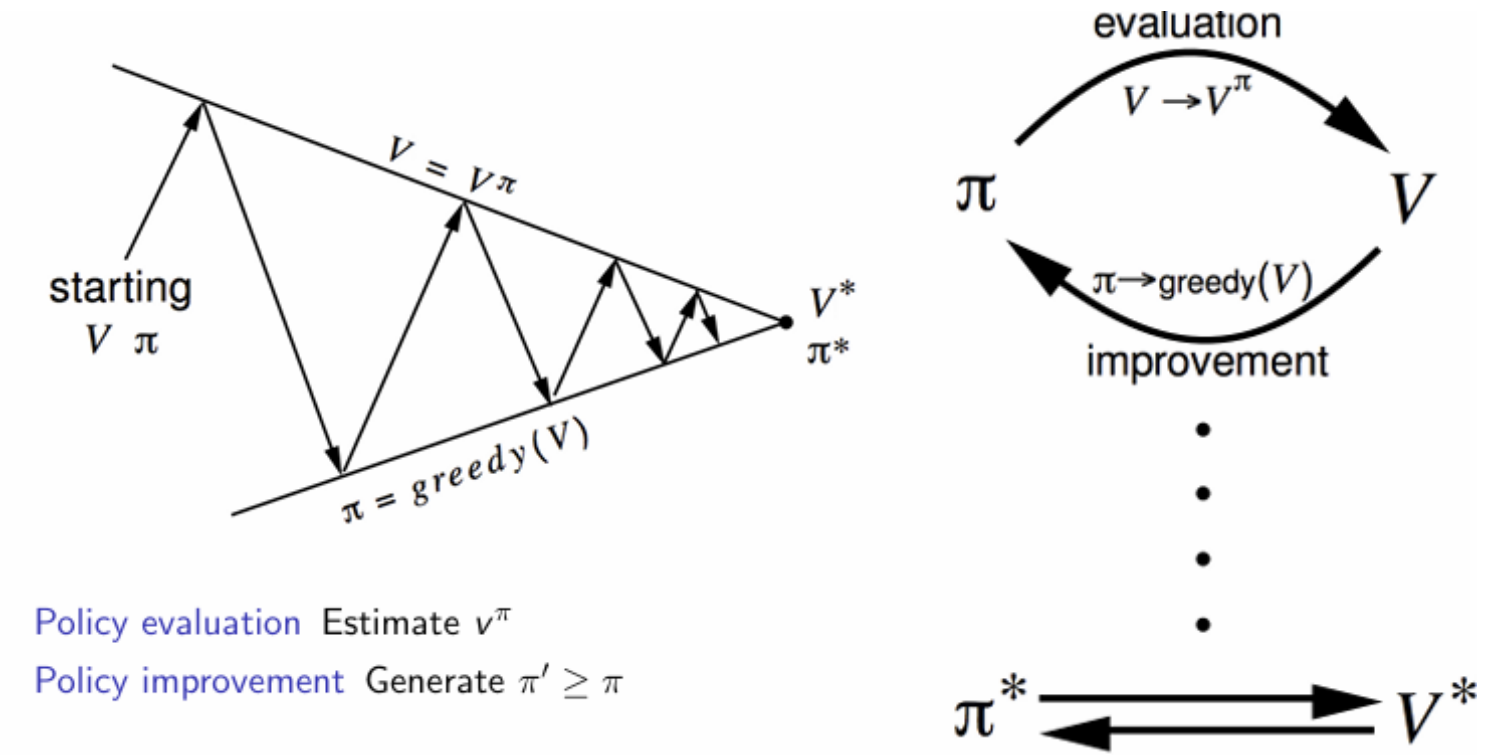


Figure: Policy Iteration algorithm view

Policy Iteration (2)

- **Policy Evaluation** compute the state-value V_π for a given policy π:

$$
\begin{aligned}
V_{t+1}(s) &= \mathbb{E}_\pi[r + \gamma V_t(s') \mid S_t = s] \\
&= \sum_a \pi(a \mid s) \sum_{s',r} P(s', r \mid s, a)(r + \gamma V_t(s'))
\end{aligned}
$$

- **Policy Improvement** Based on the value functions, Policy Improvement generates a better policy $\pi' \geq \pi$ by acting greedily: $\pi'(s) = \text{argmax}_{a \in \mathcal{A}} \mathsf{Q}_\pi(s, a)$
- The value of π' is guranteed to improve since:

$$
\begin{aligned}
Q_\pi(s, \pi'(s)) &= Q_\pi\left(s, \arg\max_{a \in \mathcal{A}} Q_\pi(s, a)\right) \\
&= \max_{a \in \mathcal{A}} Q_\pi(s, a) \geq Q_\pi(s, \pi(s)) = V_\pi(s)
\end{aligned}
$$

Policy Iteration

- **Policy Iteration(3)**: The *Generalized Policy Iteration (GPI)* algorithm refers to an iterative procedure to improve the policy when combining policy evaluation and improvement:

$$\pi_0 \xrightarrow{\text{evaluation}} V_{\pi_0} \xrightarrow{\text{improve}} \pi_1 \xrightarrow{\text{evaluation}} V_{\pi_1} \cdots \xrightarrow{\text{improve}} \pi_* \xrightarrow{\text{evaluation}} V_*$$

Value Iteration

- Value iteration algorithm applies the Bellman optimality equation.

Algorithm (Value Iteration)

- Initialize V_0
- Update: $V_{k+1} \leftarrow \max_a E[R_{t+1} + \gamma V_k(S_{t+1}) | S_t = s, A_t = a]$
- Stopping: whenever $V_{k+1}(s) = V_k(s)$ for all s, $V_k(s)$ convergence on $V^*(s)$.

- The Bellman optimality equation is equivalent to the Bellman expectation equation, adding greedy $\max$ as a policy improvement.

View in Bellman Equation

Problem	Bellman Equation	Algorithm
Prediction	Bellman Expectation Equation	Iterative Policy Evaluation
Control	Bellman Expectation Equation + (Greedy) Policy Improvement	Policy Iteration
Control	Bellman Optimality Equation	Value Iteration

Observations:

- Algorithms are based on state-value function $V_\pi(s)$ or $V^*(s)$ $\implies$ complexity $O(|A||S|^2)$ per iteration, for A actions and S states space.
- Could also apply to action-value function $Q_\pi(s,a)$ or $Q^*(s,a)$ $\implies$ $O(|A||S|^2)$ per iteration.

Monte Carlo Method

Monte Carlo Sampling

- When agents have no model, how do agents learn?
- Using sampling for estimate state value $E[G_t|S_t = s]$ or state-action value $E[G_t|S_t = s, A_t = a]$ (Monte Carlo sampling).

$$G_t^{MC} \sim E[G_t|S_t = s]$$

data collected from interaction R_1,R_2, R_3, ..., R_T

$$G_t = R_t + \gamma R_{t+1} + ... + \gamma^T R_T$$

iteration learning to update:

$$V_{n+1}(S_t) = V_n(S_t) + \alpha(G_t - V_n(S_t))$$

Monte Carlo Policy Evaluation

First-Visit Monte Carlo

$$V(s) \leftarrow \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_t^{(i)}, \quad \text{where } t \text{ is the first visit of } s \text{ in episode } i$$

Every-Visit Monte Carlo

$$V(s) \leftarrow \frac{1}{N(s)} \sum_{t \in \mathcal{T}(s)} G_t$$

$\Longrightarrow$ Monte Carlo sampling is impossible in reality, such as a continual task or a task with a long episode.

Temporal Different Learning

Temporal Update

- Disadvantage of MC: high variance, updates appear at the final episode, not suitable for a continual task.
- Temporal difference can arise from incomplete episodes, by bootstrapping.
- Temporal difference learning update after each step

$$\delta_t = (R_{t+1} + \gamma V(S_{t+1}) - V(S_t)) \text{ (temporal error)}$$

$$V_{t+1}(S_t) = V_t(S_t) + \alpha\delta_t$$

- Example algorithm: Q-learning, Sara,...

Online and Offline Learning

- **Behaviour Policy** used to interact with the environment purpose to sample the experiment.
- **Target Policy** refers to the policy we want to evaluate or improve.
- **Online Learning** is the learning process with a sample experiment and updates using a behavior policy.
- **Offline Learning** instead uses behaviour policy for sample experiment and target policy for update.

Sarsa learning

- Sarsa (state-action-reward-nextState-action) is an online learning method using temporal difference update.
- Sarsa update state-action value with a sample $s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}$:

$$Q_t(s_t, a_t) = Q_t(s_t, a_t) + \alpha(r_{t+1} + \gamma Q_t(s_{t+t}, a_{t+1}) - Q_t(s_t, a_t))$$

Q-learning

- Q-learning is an offline policy with temporal difference update.
- Q-learning update state-action value with a sample $s_t, a_t, r_{t_1}, s_{t+1}, a_{t+1}$:

$$Q_t(s_t, a_t) = Q_t(s_t, a_t) + \alpha(R_t + \gamma \arg\max_a Q_t(s_{t+1}, a) - Q(s_t, a_t))$$

- Q-learning is a model-free control algorithm without any dynamic model of the environment. Try to estimate the expected state-action value by temporal sampling.

Overestimation Q-learning

- **Problem Overestimation with Q-learning**: Q-learning overestimates because it uses the same state-action value to select and evaluate.
- proof: Consider a state s_t with discrete set action $\{a_1, ..., a_k\}$:

$$\hat{Q}(s_t, a_i) = Q^*(s_t, a_i) + \epsilon_i, \text{with } E[\epsilon_i] = 0$$

$$E[\max(\{\hat{Q}(s_t, a_i) | i \in \bar{1, k}\})] = E[\max\{Q^*(s_t, a_i) + \epsilon_i | i \in \bar{1, k}\}]$$

$$\geq^{Jesen's\ Inequality} \max(\{E[Q^*(s_t, a_i) + \epsilon_i] | i \in \bar{1, k}\})$$

$$\geq \max(\{Q^*(s_t, a_i) | i \in \bar{1, k}\})$$

$\Longrightarrow$ operator max gives Q-learning in an overestimate.

Double Q-learning

- Double Q-learning introduced two Q-values in select and evaluate, which solves the problem of overestimating in Q-learning.
- Double Q-learning update:

$$Q_t^{(1)}(s_t, a_t) = Q_t^{(1)}(s_t, a_t) + \alpha(r_{t+1} + \gamma \arg\max_a Q_t^{(2)}(s_t, a) - Q_t^{(1)}(s_t, a_t))$$

$$Q_t^{(2)}(s_t, a_t) = Q_t^{(2)}(s_t, a_t) + \alpha(r_{t+1} + \gamma \arg\max_a Q_t^{(1)}(s_t, a) - Q_t^{(2)}(s_t, a_t))$$

- Double Q-learning randomly select $Q^{(1)}$ or $Q^{(2)}$ to update and final policy combined by optimal $Q^{(1)}$ and $Q^{(2)}$.

Importance Sampling in Off-policy Learning

Off-policy learning

- Off-policy learning means learning with policy π_t but the experiment data is generated by policy π_b.
- Q-learning is an example of off-policy learning.
- Q-learning just one transition update, however, for long transitions or trajectory estimate value under difference policy can lead to distribution shift.

Importance sampling corrections

- Importance sampling is used to estimate the expectation of a distribution under the data from a different distribution.
- Importance sampling expression: given a random variable X generated from distribution $p(x)$ and a function $f(x)$, estimate the expectation of $f(x)$ under a different distribution $g(x)$.

$$E_{x\sim g}[f(x)] = \sum g(x)f(x)$$

$$= \sum p(x)\frac{g(x)}{p(x)}f(x)$$

$$= E_{x\sim p}[\frac{g(x)}{p(x)}f(x)]$$

- Intuition: importance sampling is a weighting method on data by the ratio s$\frac{g(x)}{p(x)}$.

Importance Sampling for off-policy Monte Carlo Method

- Given a trajectory $\tau_t = \{s_t, a_t, r_{t+1}, s_{t+1}, ...\}$ generated from behaviour policy π_b.
- we need to estimate the expectation of return value $E[G_t|s \sim p(s), a \sim \pi_t(a|s)]$.
- Importance sampling:

$$E[G_t|s \sim p(s), a \sim \pi_t(a|s)] = E[\frac{p(\tau_t|\pi_t(a|s))}{p(\tau_t|\pi_b(a|s))} G_t|s \sim p(s), a \sim \pi_b(a|s)]$$

$$= E[\frac{p(s_t)\pi_t(a_t|s_t)p(s_{t+1})\pi_t(a_{t+1}|s_{t+2})...}{p(s_t)\pi_b(a_t|s_t)p(s_{t+1})\pi_b(a_{t+1|s_{t+2}})...} G_t|s \sim p(s), a \sim \pi_b(a|s)]$$

$$= E[\frac{\pi_t(a_t|s_t)\pi_t(a_{t+1}|s_{t+1})...}{\pi_b(a_t|s_t)\pi_b(a_{t+1}|s_{t+1})...} G_t|s \sim p(s), a \sim \pi_b(s|a)]$$

- Monte carlo importance sampling update:
$Q_t(s_t, a_t) = Q_t(s_t, a_t) + \alpha(\frac{\pi_t(a_t|s_t)\pi_t(a_{t+1}|s_{t+1})...}{\pi_b(a_t|s_t)\pi_b(a_{t+1}|s_{t+1})...} G_t - Q_t(s_t, a_t))$

N-step Bootstrap

Bias and Variance Trade-off

- TD can immediately learn without wait to the terminal state. However, by temporal update, TD can lead to a biased estimate.
- Monte Carlo instead update when finishing the trajectory. By updating on the long horizon, the MC method has high variance.
- we can combine TD and MC by introducing the N-step bootstrap method.

N-step Method

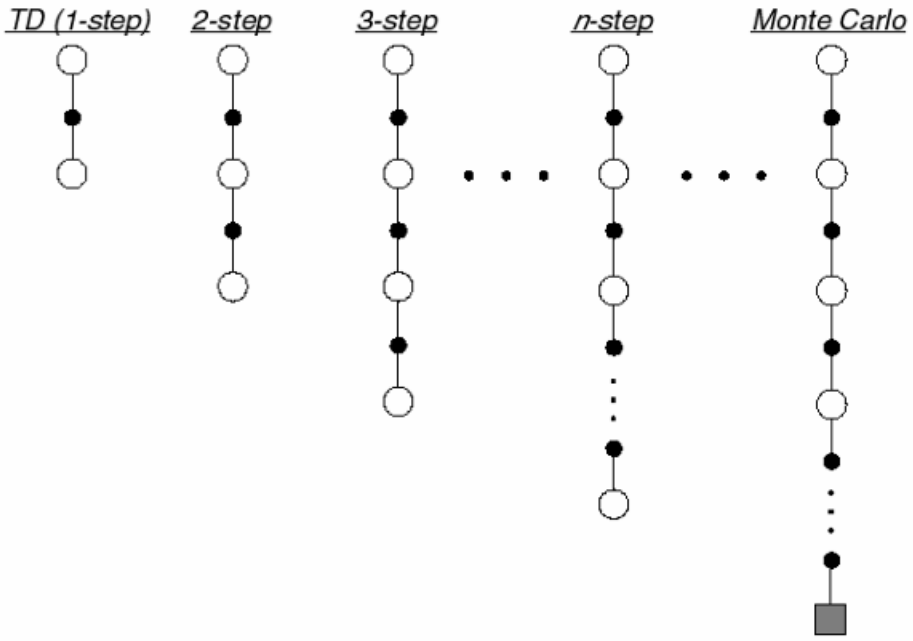


Figure: N-step update diagram.

- The N-step state return value is defined by

$$G_t^{(n)} = r_{t+1} + \gamma r_{t+2} + ... + \gamma^{n-1} r_{t+n} + \gamma V(s_{t+n})$$

- The N-step temporal difference learning

$$V(s_t) = V(s_t) + \alpha(G_t^n - V(s_t))$$

λ-return

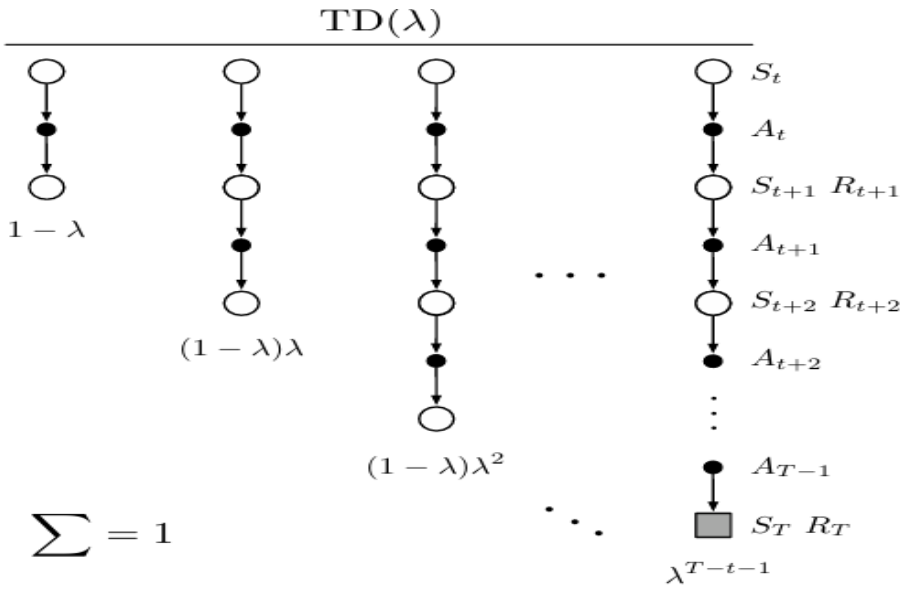


Figure: The backup diagram for λ-return

- Note: The bootstrap concept in statistics is a method of resampling data. However, in RL, the bootstrap refers to reusing the previous estimate value.
- λ-return is a mixture of N-step return

$$G_t^{\lambda} = (1-\lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_{t:t+n} + \lambda^{T-t-1} G_t$$

λ-return

- where $G_{t:t+n}$ refers to the n-step value bootstrap at the state n, and G_t is the full return value.
- λ large concentrate value near the bootstrap step at
- λ small concentrate value on far bootstrap step.
- λ-return can balance between bias and variance. However, it also requires a full trajectory to learning, leading to delayed learning.
- How to solve the weakness of delayed learning and still balance between bias and variance?

Eligibility Traces

Forward View

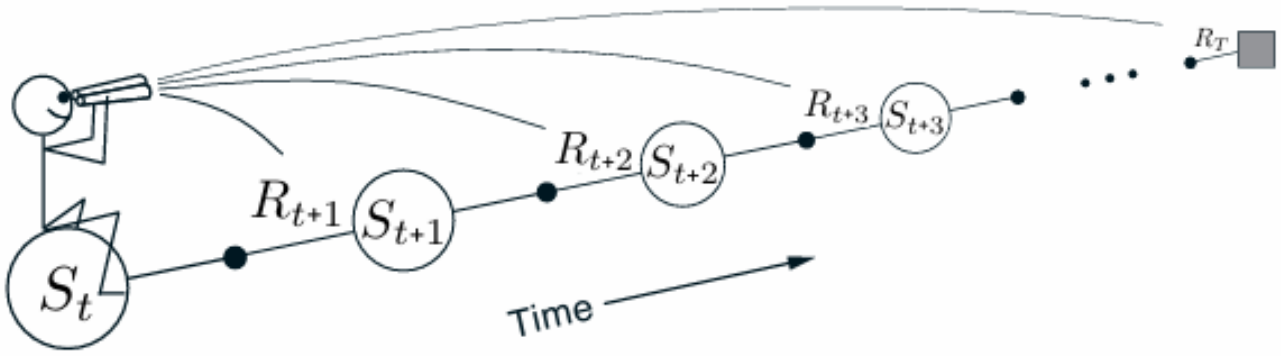


Figure: The forward view in the update of state by looking forward to future rewards and states

- recall: Monte Carlo or λ-return follow in forward view to update value. It can not apply to online learning and lead to a delay in learning.
  $\Longrightarrow$ Can we transfer from the forward view to the backward view (from current state to past states)?

Eligibility Traces for Monte Carlo

- The MC errors can be rewritten as a sum of TD errors:

$$
\begin{aligned}
G_t - V(s_t) =& r_{t+1} + \gamma G_{t+1} - V(s_t) && (1)\\
=& (r_{t+1} + \gamma V(s_{t+1}) - V(s_t)) + \gamma(G_{t+1} - V(s_{t+1})) && (2)\\
=& \delta_t + \gamma \delta_{t+1} + \gamma^2 (G_{t+2} - V(s_{t+2})) && (3)\\
=& \sum_{k=t}^{T} \gamma^{k-t} \delta_k && (4)
\end{aligned}
$$

Eligibility Traces for Monte carlo

- Total sum of Monte Carlo update:

$$\Delta V(s) = \alpha \sum_{t=0}^{T-1} (G_t - V(s_t)) \tag{5}$$

$$= \alpha \sum_{t=0}^{T-1} (\sum_{k=t}^{T-1} \gamma^{k-t} \delta_t) \tag{6}$$

$$= \alpha \sum_{k=0}^{T-1} (\sum_{t=0}^{k} \gamma^{k-t} \delta_t) \; (\sum_{i=0}^{m} \sum_{j=i}^{m} z_{ij} = \sum_{j=0}^{m} \sum_{i=0}^{j} z_{ij}, \text{ backward view }) \tag{7}$$

$$= \alpha \sum_{k=0}^{T-1} \delta_k \sum_{t=0}^{k} \gamma^{k-1} \tag{8}$$

Eligibility Traces for Monte Carlo

- The total sum of monte carlo become:

$$\Delta V(s) = \alpha \sum_{k=0}^{T-1} \delta_k e_t$$

- where e_t is the eligibility trace with a recursive formula:

$$e_t = \sum_{t=0}^{k} \gamma^{k-t} = \gamma \sum_{t=0}^{k-1} \gamma^{k-t-1} + 1 \tag{9}$$

$$= \gamma e_{t-1} + 1 \tag{10}$$

- Up to now, the Monte Carlo update for each step can rewrite

$$V_{t+1}(s_t) = V_t(s_t) + \alpha \delta_t e_t$$

- Monte Carlo update becomes a backward view and can apply for online learning

Eligibility Traces for λ-return

- The λ-return error :

$$G_t^\lambda - V(s_t) = (1-\lambda)\sum_{i=1}^{T-t-1}\lambda^{i-1}G_{t:t+i} + \lambda^{T-t-1}G_t - V(s_t)$$

$$= (1-\lambda)G_{t:t+1} + (1-\lambda)\sum_{i=2}^{T-t-1}\lambda^{i-1}G_{t:t+i} + \lambda^{T-t-1}G_t - V(s_t)$$

$$= (1-\lambda)(r_{t+1} + \gamma V(s_{t+1})) + (1-\lambda)\sum_{i=2}^{T-t-1}\lambda^{i-1}(r_{t+1} + \gamma G_{t+1:t+i})$$

$$+ \lambda^{T-t-1}(r_{t+1} + \gamma G_{t+1}) - V(s_t)$$

$$= (G_{t:t+1} - V(s_t)) + [(1-\lambda)\sum_{i=1}^{T-t-1}\lambda^{i-1} + \lambda^{T-t-1} - 1]r_{t+1}$$

$$+ \lambda\gamma[(1-\lambda)\sum_{i=2}^{T-t-1}\lambda^{i-1}G_{t+1:t+i} + \lambda^{T-t-2}G_{t+1} - V(s_{t+1})]$$

Eligibility Traces for λ-return

- The λ-return error can be rewritten as a sum of temporal difference error:

$$G_t^\lambda - V(s_t) = \delta_t + [\underbrace{(1-\lambda)\sum_{i=1}^{T-t-1}\lambda^{i-1} + \lambda^{T-t-1}}_{1} - 1]r_{t+1}$$

$$+ \lambda\gamma[(1-\lambda)\sum_{k=1}^{T-(t+1)-1}\lambda^{k-1}G_{t+1:t+1+k} + \lambda^{T-(t+1)-1}G_{t+1} - V(s_{t+1})]$$

$$, (\text{ let } t = k+1)$$

$$= \delta_t + \lambda\gamma(G_t^\lambda - V(s_{t+1}))$$

$$= \delta_t + \lambda\gamma\delta_{t+1} + (\lambda\delta)^2 + ... + (\lambda\delta)^{T-1} = \sum_{k=0}^{T-1}(\lambda\gamma)^k\delta_{t+k}$$

Eligibility Traces for λ-return

- Similarity with the previous section in Monte Carlo. The eligibility traces for λ-return :

$$e_t = \lambda\gamma e_{t-1} + 1$$

$$V_{t+1}(s_t) = V_t(s_t) + \alpha\delta_t e_t(s_t)$$

- Monte Carlo is a special case with $\lambda = 1$.
- we also apply eligibility traces for other methods in approximation, which is mentioned in the next section.

Exploration and Exploitation

Exploration and Exploitation

- A challange question in reinforcement learning: at a state, agent must to exploit action with current best reward, or explore new action with hoping get more expected reward.
  - exploitation: take action by greedy strategy
  - exploration: try a new action to explore more

$\longrightarrow$ This question is called trade-off exploration and exploitation.

Introduction to Multi-Arm Bandit (MAB)

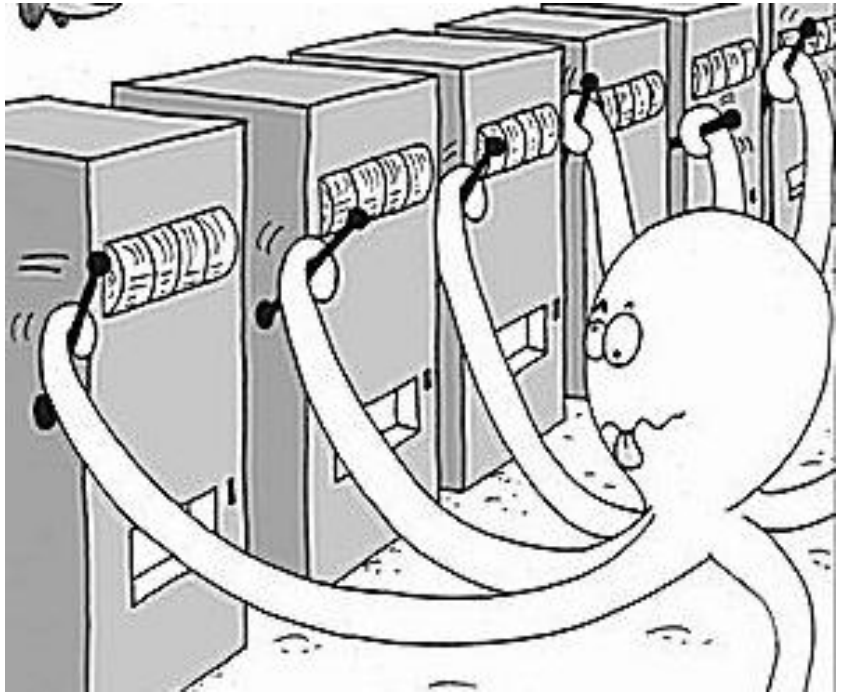


Figure: K-arm Machine as an example of bandit problem

- MAB is a decision-making problem with one state and a certain action like arms machine in the figure.
- MAB is a theorical framework to evaluate trade-off exploration and exploitation.

MAB Problem Formulation

- MAB has many problem setting, however Stochastic MAB is one of the most general settings.

Stochastic Multi-Armed Bandits

Consider a K-armed stochastic bandit problem where each arm has a fixed reward distribution over time.

- Action space: $\mathcal{A} = \{a_1, a_2, \ldots, a_K\}$
- R_{t,a_i} is the reward received from arm a_i at time t
- The reward distribution of each arm is often assumed to belong to the set of σ-sub-Gaussian distributions, satisfying:

$$\mathbb{E}\left[\exp(\lambda X)\right] \leq \exp\left(\frac{\lambda^2\sigma^2}{2}\right), \quad \forall \lambda \in \mathbb{R}$$

Multi-Arm Bandit Approach

Frequentist

- No prior assumption on the reward distribution of each arm: $\theta_1, \ldots, \theta_K$ are unknown.
- Estimate each arm's mean reward using confidence bounds: $P(\mu_a \leq \text{UCB}_a(t)) \geq \gamma$
- Objective: minimize expected cumulative regret $R_n(\theta) = \mathbb{E}\left[\sum_{t=1}^{n}(\mu^* - \mu_{A_t})\right]$
- **Example:** UCB (selects actions via optimism principle)

Bayesian

- Assumes a prior distribution over parameters of each arm: $\theta_a \sim P_a$
- Updates posterior distribution with observations: $P(\hat{\theta}|D) \propto P(D|\hat{\theta})P(\hat{\theta})$
- Objective: minimize Bayes risk $\text{Risk}_n = \mathbb{E}[R_n(\theta)]$
- **Examples:** Thompson Sampling, Bayes-UCB

Challenges in Two Approaches

Frequentist

- Performance heavily depends on the construction of tight, high-probability confidence intervals.
  $\Rightarrow$ Hard to design a good bound in a complex environment.

Bayesian

- Thompson Sampling is efficient when prior and posterior distributions are conjugate.
- Posterior sampling becomes intractable when parameters are complex.
  $\Rightarrow$ Approximate inference methods are used, but they require more computational resources.

Planning and Dynamic Model

Model of Environment

- In the previous section, we introduced several methods in the model-free approach, such as Monte Carlo and temporal difference, that did not require knowledge of the environment's model.
- The model-free learn directly from the experiment by estimating the value functions.
- The model-based learn a model from experience and plans from the learned model.

What is the Model Environment?

- The model of the environment can understand as the transition model from the current state to the next state, called the Dynamic Model.

Definition (Dynamic Model)

A model $M_\theta(S, A)$ is an approximate representation of an MDP $< S, A, P_\theta, \gamma >$, with P_θ being a combined model of transition and reward.

$$r_{t+1}, s_{t+1} \sim P_\theta(r, s^{'} | s_t, a_t)$$

- The model mentioned in the definition follows a stochastic case. We can use a deterministic model for the environment.

$$f(s_t, a) = s_{t+1}$$

Model Learning

- Why we need to learning dynamic model?
  - The environment is not deterministic, learning model to reason aboutt the uncertainty of environment.
  - Interaction with real environment or simulation are expensive. Model-base can have reduce the interaction with the environment.
- We can learning a model by supervised learning.
  - Collect the transition sample from experience$\{s_1, a_1, r_1, ..., s_t\}$.
  - Then, estimate model M_θ from the experience by supervise learning.

$$s_1, a_1 \rightarrow r_2, s_2$$

$$...$$

$$s_{t-1}, a_{t-1} \rightarrow r_t, s_t$$

  - This model is kind of deterministic model.

Disadvatage of Model-based

- **Disavantage**
  - The efficiency of the model-based method depend much more on the process of learning the model.
  - Two error approximates learning, from learning dynamic model, and then learn to find optimal policy.

Dyna-Q Algorithm

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Do forever:
- (a) $s \leftarrow$ current (nonterminal) state
- (b) $a \leftarrow \varepsilon$-greedy(s, Q)
- (c) Execute action a; observe resultant state, s', and reward, r
- (d) $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$
- (e) $Model(s,a) \leftarrow s', r$ (assuming deterministic environment)
- (f) Repeat N times:
  - $s \leftarrow$ random previously observed state
  - $a \leftarrow$ random action previously taken in s
  - $s', r \leftarrow Model(s,a)$
  - $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$

Figure: Pseudo-code of Dyna-Q learning algorithm. Dyna-Q is a combination of learning and planning.

Planning

- In the concept of artificial intelligence, Planning refer to any computational process that takes a model as input and produces a policy for interating with the modeled environment:

$$\text{model} \longrightarrow^{Planning} \text{policy}$$

- Planning alway require have a model of environment.
- Planning also estimate the value function like learning.

Monte Carlo Tree Search

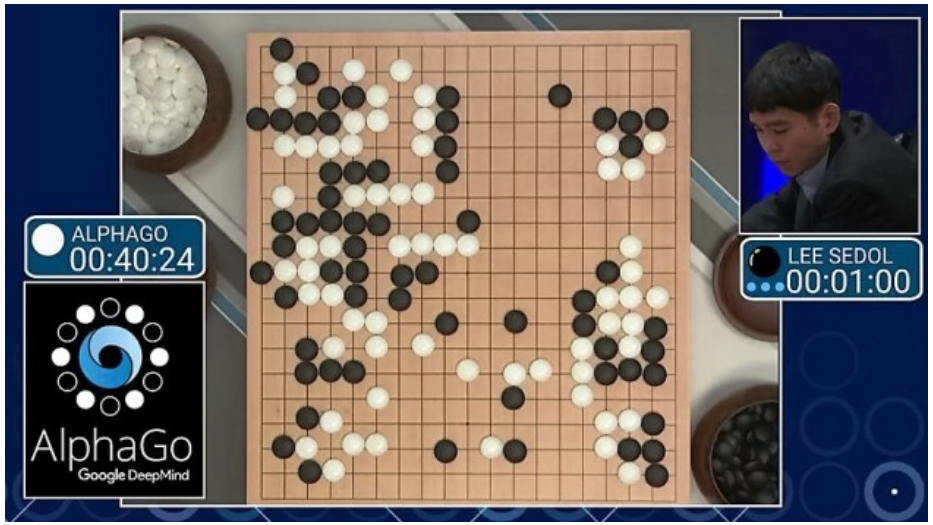


Figure: AlphaGo (Google DeepMind) competing against professional Go player Lee Sedol

- MCTS is a planning algorithm for MDPs, best known for its success in AlphaGo developed by Google DeepMind.
- MCTS plans a sequence of actions by building a search tree, where each state–action decision is formulated as a **MAB** problem to balance exploration and exploitation (select actions like select bandit arms).

Monte Carlo Tree Search

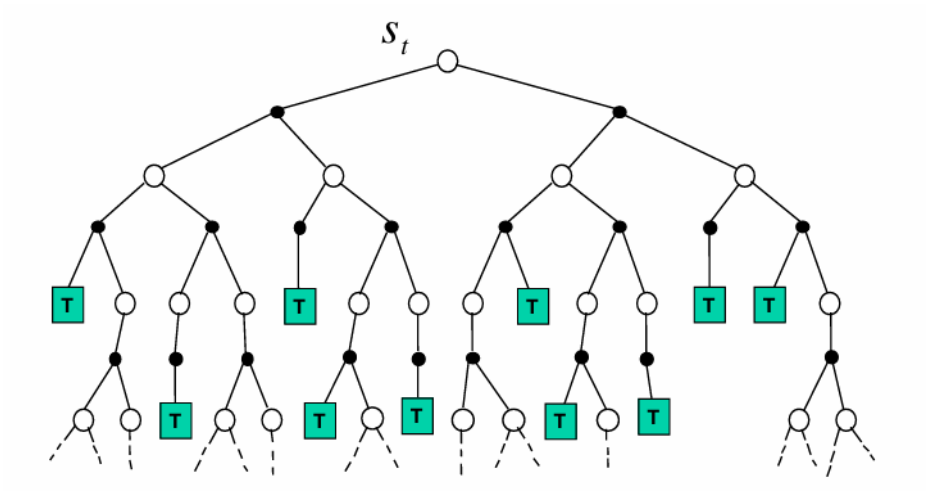


Figure: A Markov Decision Process starting at state s_t, with discrete action and state space.

- MCTS builds a search on a MDP by using statistical evidence, and then gives a sequential decision through the tree.
- We consider a MCTS as a tree with multiple states, with each state representing a bandit problem.
- MCTS algorithm has 4 phases in each update, including selection, expansion, simulation, and backpropagation.

Monte Carlo Tree Search

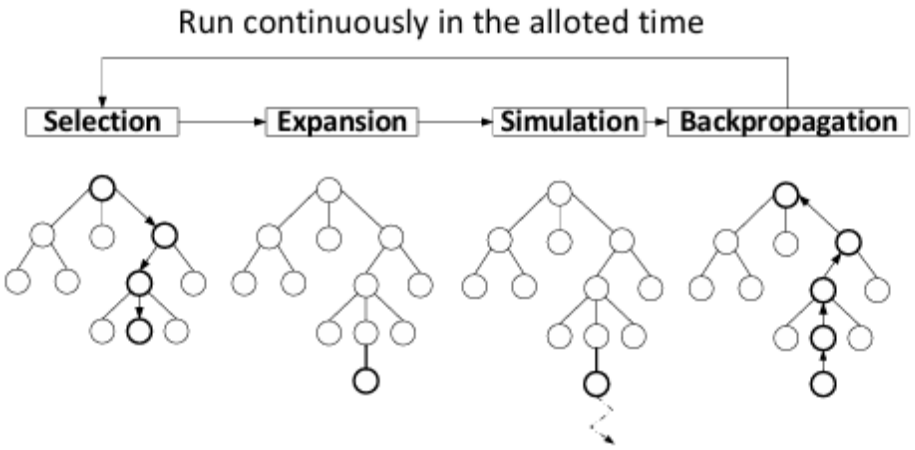


Figure: Monte Carlo Tree Search phases.

- **Selection Phase**: The algorithm starts from the initial state traversal to reach a leaf node of the tree. The selection follows a tree policy based on $Q(s, a)$ at each state node.

$$\textbf{UCT Tree Policy} : a^* = \arg\max_{a \in A(s)} \{Q(s,a) + C\sqrt{\frac{\ln N(s)}{N(s,a)}}\}$$

  - Tree policy is a kind of upper bound in MAB, and is based on the optimistic principle.

Monte Carlo Tree Search

- **Expansion**: When the selection phase reaches the leaf node, the tree must expand a new node. However, when expansion reaches the terminal state, which is extremely rate, then the current iteration skips directly to backpropagation.
- **Simulation**: perform a comlete trajectory from the expanded node at the expansion phase. This phase is also called rollout, and it is a Monte Carlo simulation to compute Q-value without expanding any new node.

$$Q(s_p, a) = \frac{1}{K} \sum_{k=1}^{k} G_t^k$$

Monte Carlo Tree Search

- **Backpropagation**: Propagate the score of the simulation phase to ancetor node. The backpropagation uses the Bellman equation to update the Q-value:

$$G \leftarrow r + \gamma G$$

$$Q(s,a) \leftarrow Q(s,a) + \frac{1}{N(s,a)}[G - Q(s,a)]$$

- Link tutorial in MCTS

Thank You for Watching !!!